# How Spotify and Netflix Build Their ML Pipelines

Scaler School of Technology | ML System Design and LLMOps | Week 5 — Session 2 | Deep-Dive Case Study

**This session:** You designed a music recommendation system in W5S1. Today we compare your designs to what Spotify and Netflix actually built. Bring your W5S1 worksheet — you will find that the industry converged on simpler, not more complex, architectures.

## The Two Systems We Are Studying

### Spotify Discover Weekly

**Scale:** 30 million users, every Monday
**Output:** 30-song personalised playlist
**Cadence:** Weekly batch job
**Key tech:** word2vec on playlist data
**Cold-start:** CNN audio embeddings for new songs

### Netflix Recommendations

**Scale:** 200+ million members
**Output:** Ranked content rows
**Cadence:** Candidate model daily, ranker hourly
**Key tech:** Two-tower neural network + ANN
**North star:** Streaming hours, not CTR

## Compare Your W5S1 Design: Five Dimensions

| Dimension | Spotify | Netflix | Your design |
|---|---|---|---|
| Model architecture | word2vec on playlists + audio CNN | Two-tower neural network | *(fill in)* |
| Serving strategy | Weekly batch, pre-computed | Candidate daily, ranker hourly | *(fill in)* |
| Cold-start | Audio CNN for new songs | Feature engineering (recency, genre) | *(fill in)* |
| Evaluation north star | Implicit — stream completion rates | Streaming hours (not CTR) | *(fill in)* |
| Scale of freshness | Weekly update cycle | Hourly ranker retraining | *(fill in)* |

**Key pattern before we dive in:** Both Spotify and Netflix use architectures you have seen before — word2vec, neural networks, ANN search. The sophistication is not in the model family; it is in *how the pipeline is engineered* around those models. Freshness strategy, cold-start handling, and evaluation design are where production systems differ from classroom projects.

# Spotify Discover Weekly: The Pipeline Deep-Dive

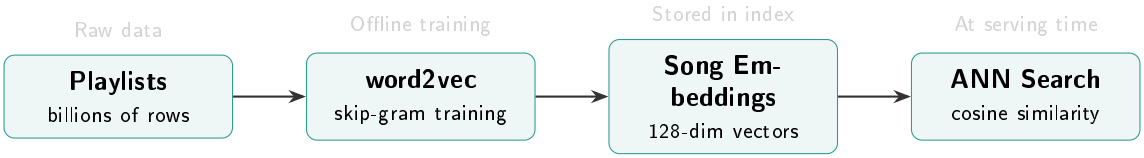
Published engineering blog · 30M users · Weekly batch · Three-model ensemble

## Model 1 | Collaborative Filtering via word2vec on Playlists

**The key insight:** Treat playlists as sentences. Treat songs as words. Apply word2vec. Songs that co-occur in many playlists end up close together in the embedding space — regardless of genre label, audio features, or explicit tags. The playlist context is the signal.

**Mechanism step by step:**

1. Aggregate all user playlists across the platform. Each playlist is treated as a "sentence" — an ordered sequence of song IDs.
2. Train a skip-gram word2vec model. For each song ("word"), learn a 128-dimensional embedding such that songs appearing in similar playlist contexts have high cosine similarity.
3. Example: If users A and B both have Radiohead and Thom Yorke in their playlists, and user A also has Portishead, then Portishead's embedding will be close to Radiohead's. Portishead is then a reasonable recommendation for user B.
4. Result: 40+ million song embeddings. Songs close in this space share *cultural context*, not just acoustic similarity.




## Model 2 | Content-Based Filtering via Audio CNN

**What it does:** For every song in the catalogue, pass raw audio through a convolutional neural network trained to predict audio features (genre tags, mood, instrumentation). The final hidden layer produces a song embedding based purely on acoustic content — no playlist co-occurrence required.

**Why it matters:** New songs cannot appear in word2vec embeddings because they have not yet appeared in any user playlists. The audio CNN gives every new song an embedding from day 1. This is Spotify's cold-start solution for new music.

**Architecture:** Input is a log-mel spectrogram. The CNN extracts timbral, rhythmic, and structural features. Multiple convolutional layers with max-pooling reduce the spectrogram to a fixed-length embedding.

## Model 3 | NLP on Text: Cultural Context Embeddings

**What it does:** Crawl the internet for text about artists and songs — lyrics, reviews, blog posts, social media posts, music journalism. Apply word2vec to this text corpus. The resulting embeddings capture *cultural meaning*: the community and discourse around an artist.

**Example:** "Post-rock" artists cluster together in this embedding space not because their playlists overlap, but because music journalists describe them using the same vocabulary.

**Why three models?** Each embedding space captures a different aspect of musical similarity. Combining them makes the final recommendation more robust — a song can be similar to your history via playlist co-occurrence, acoustic content, *or* cultural discourse.

## Combining | Ensemble: Three Embedding Spaces into One Score

**At recommendation time:**

1. Compute user's listening history embedding (average of all song embeddings in their history)
2. For each candidate song, compute cosine similarity across all three embedding spaces
3. Weighted combination of the three similarity scores
4. Rank by combined score, return top 30

**Timing:**

- Runs as a **weekly batch job** on Hadoop + Spark
- Every Monday: all 30M Discover Weekly playlists regenerated
- Total time: ~30 minutes for all users (at Hadoop scale)
- New songs appearing in playlists this week will appear in next Monday's embeddings

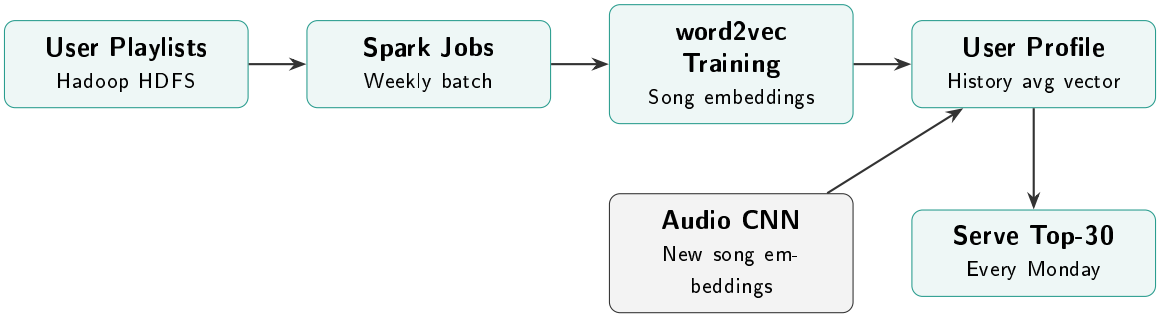
## Spotify: Cold-Start Problem for New Songs

**The problem:** A song released today cannot appear in word2vec embeddings because it has had zero playlist co-occurrences. With >40,000 songs released per day on Spotify, this is not an edge case.

| Song age | Available signal | Embedding source |
|---|---|---|
| 0–7 days | No playlist data | Audio CNN only |
| 7–30 days | Limited playlist co-occurrences | Audio CNN + partial collaborative |
| 30+ days | Rich playlist data | Full three-model ensemble |

**Design implication:** Spotify engineers maintain *two separate embedding indices*: one for the full collaborative signal (updated weekly) and one for audio-only embeddings (updated as new songs are uploaded). The serving layer queries both and blends the results. This is a deliberate **separation of concerns** in the pipeline — not an afterthought.

## Spotify: What This Pipeline Looks Like End-to-End

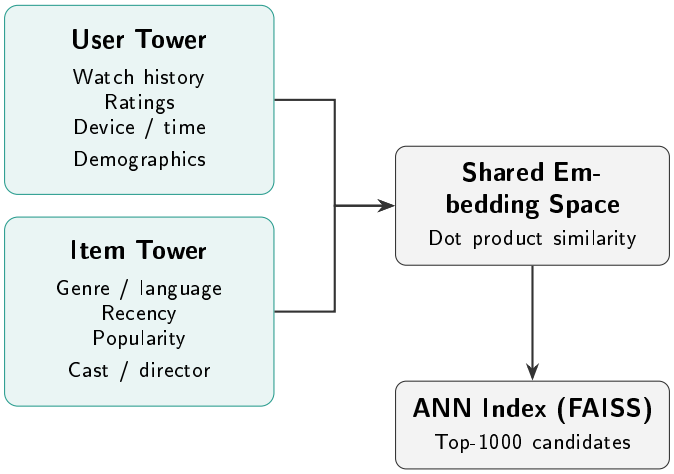


# Netflix: Two-Tower + Ranking Architecture

Published research · 200M+ members · Candidate model daily, ranker hourly · Streaming hours as north star

## Stage 1 | Candidate Generation: Two-Tower Neural Network

**Architecture:** A two-tower neural network with a user tower and an item tower. Both towers are trained jointly on watch events. At inference time, Approximate Nearest Neighbour (ANN) search retrieves the top 1,000 candidates for a given user.




**Training:** Trained on watch events (implicit feedback). A positive pair is (user, title they watched >50% of). Negative pairs are randomly sampled titles. The loss function (softmax or in-batch negatives) pushes watched items closer to the user embedding and unwatched items further away.

**Retrained:** Daily. The ANN index is rebuilt after each training run.

## Stage 2 | Ranking: Separate Neural Network with Rich Features

**Why a separate ranker?** The two-tower model must be simple enough to build an ANN index over all items. The ranker has no such constraint — it runs over only 1,000 candidates and can use expensive, interaction-level features that cannot be pre-computed per-item.

**What the ranker optimises:**

- Probability of play (will the user click?)
- Expected play duration (will they watch >75%?)
- Post-play survey score (did they rate it positively after watching?)

**Retrained:** Hourly. Netflix deliberately uses a high-frequency retraining schedule for the ranker to capture within-day context changes (e.g., a user browsing at 11pm on a Friday has different intent than at 8am on a Tuesday).

**Feature engineering at Netflix:**

| Feature category | Examples |
|---|---|
| User history | Watch time per genre, ratings given, completion rates, time since last watch |
| Context | Device type (TV vs. mobile), time of day, day of week, country |
| Item features | Genre, language, recency of release, average rating across all users |
| Interaction (u × i) | Has user watched other titles by this director? Similar genre completion rate? |

## Netflix: Multi-Objective Evaluation

**The Netflix insight:** Maximising click-through rate (CTR) alone produces clickbait. Netflix explicitly optimises for **long-term member satisfaction**, not just immediate engagement. A model that recommends a mediocre show that a user starts and abandons is worse than one that recommends fewer but higher-quality matches.

| Metric type | Metric | Why Netflix uses it |
|---|---|---|
| Offline | NDCG@K | Measures ranking quality on held-out watch history |
| | AUC-ROC | Binary play prediction quality |
| Online | **Streaming hours** (north star) | Did users actually watch more? Direct business metric |
| | Post-play survey | Did users report enjoying what they watched? |
| | 30-day retention | Are members still subscribing? Captures long-term satisfaction |

**Why not CTR?** CTR can be gamed. A recommendation system that shows mystery thumbnails maximises clicks but does not maximise satisfaction. Netflix's engineering blog explicitly notes that their north star shifted from CTR to *streaming hours* and then to *member satisfaction surveys* as they learned that initial clicks do not predict long-term retention.

## Key Lessons: Student Design vs. Production Reality

| Common student design | What Spotify / Netflix built | The lesson |
|---|---|---|
| Transformers or deep learning | word2vec (Spotify) and two-tower MLP (Netflix) | Production favours proven, scalable architectures over the latest research |
| One model for everything | Candidate generation + separate ranker | Separation of concerns is not optional at 200M users |
| Train once, serve forever | Spotify: weekly; Netflix: hourly ranker | Freshness strategy is a first-class architectural decision |
| Cold-start as afterthought | Dedicated audio CNN pipeline at Spotify | Cold-start affects every new song and new user — design for it explicitly |
| Optimise CTR | Netflix: streaming hours + post-play surveys | Evaluation design determines what your model actually learns |

## Scale Forces Separation of Concerns

At 200M users and 100M+ catalogue items, the two-stage architecture (candidate generation + ranking) is not an optimisation — it is the only architecturally feasible option. You cannot rank all items at serving time under a 100ms latency constraint. This is why Spotify, Netflix, YouTube, TikTok, and Amazon *all* use this pattern. The pattern you designed in W5S1 was correct.